

Automation and Programmability

CCNA 200-301 V1.1

MODULE 9

Topics

9.1 Network Automation Overview

9.2 Software Defined Networks

9.3 JSON

9.4 REST APIs

9.5 Management Mechanisms

9.6 AI in Network Operations

9.7 Review



9.1 Network Automation Overview

CCNA 200-301 v1.1

Module 9

Network Management Automation

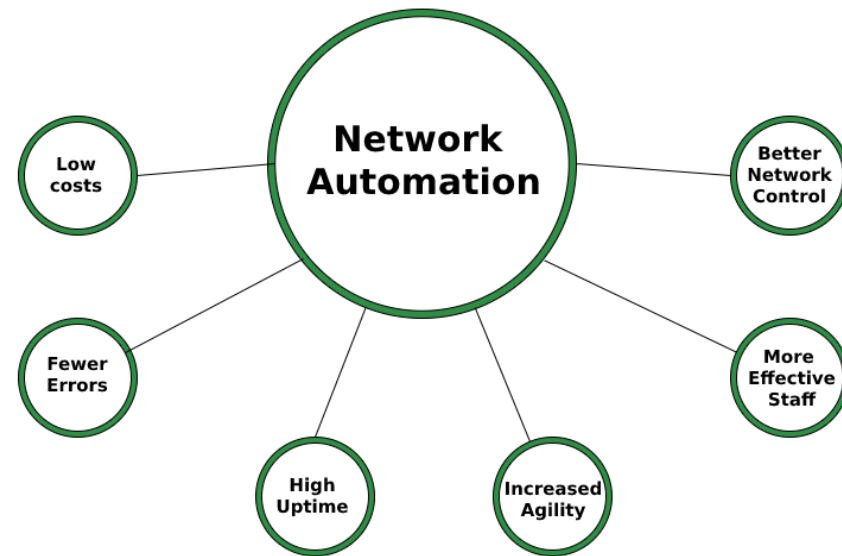
Infrastructure as Code

Network Management Automation

The use of software and tools to automate the configuration, monitoring, management, and maintenance of network devices and infrastructure

The goal is to improve efficiency, reduce human errors, enhance network performance, and ensure consistent application of policies across the network

- Faster configuration
- Real-time monitoring
- Easier expansion
- Optimized resource usage
- Rapid threat response
- Compliance



Infrastructure as Code (IaC)

Managing and provisioning computer data centers through machine-readable definition files

- Rather than physical hardware configuration or interactive configuration tools
- Most effective when underlying infrastructure is IaaS or PaaS

Separates configurations, policies, profiles, scripts, and templates from the hardware or software on which they are deployed

- These items can then be stored, shared, revised, and applied in the same manner as code

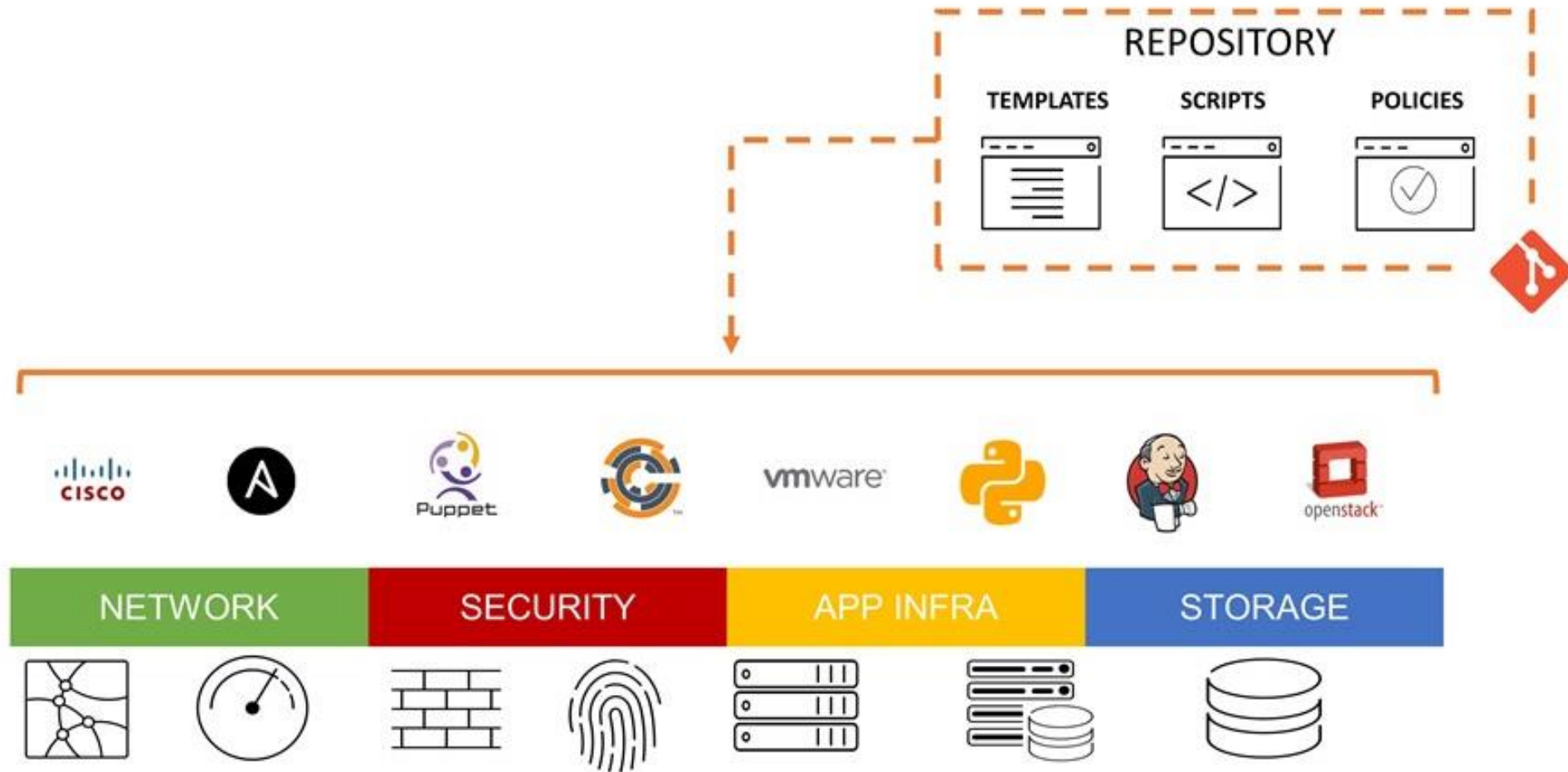
Utilizes the core best practices of DevOps

- version control, virtualized tests, continuous monitoring

Your infrastructure is treated the same way that any other code would be



Infrastructure as Code Example



9.2 Software Defined Networks

CCNA 200-301 v1.1

Module 9

Traditional Networks

Software Defined Networks

Control and Data Planes

Northbound / Southbound APIs

SDN Overlay - Underlay

Fabric

Traditional Networks

Traditional networks consist of multiple individual network devices, such as routers, switches, and firewalls

Control and data planes are tightly integrated into each network device

Each device is configured and managed separately

Configuration and policy enforcement are done manually on each device

Managing a large network can be complex and time-consuming

Policies and configurations are often static and hard to change



Software Defined Networks (SDN)

AKA Controller-based Networking

Uses a separate software controller to manage multiple network devices

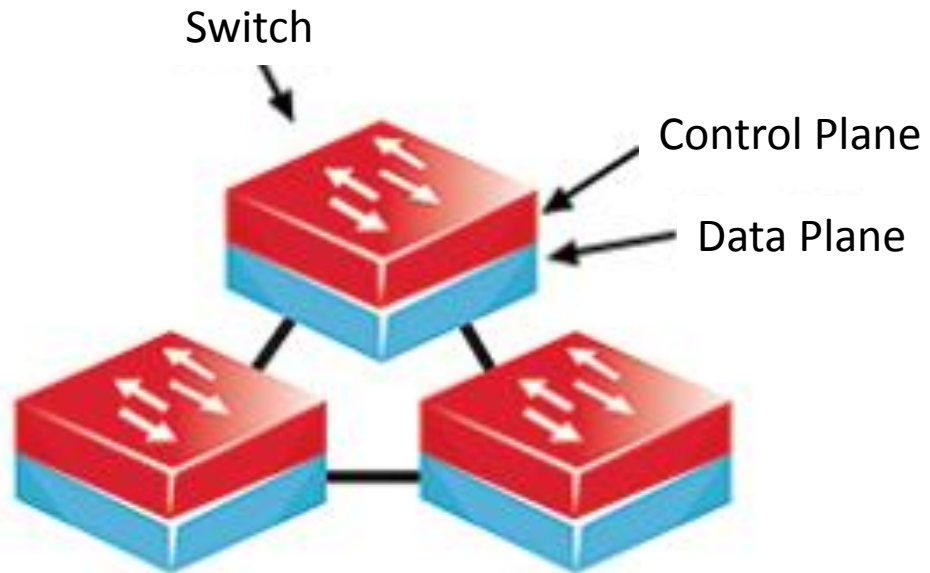
- Configurations are dynamically and automatically applied
- Policy is enforced from a single location

Easier to manage large and complex networks

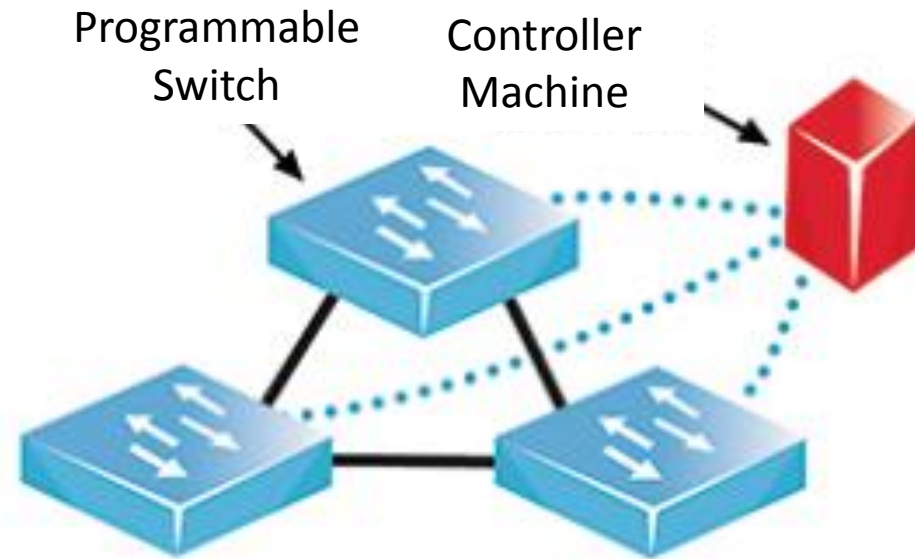


Traditional vs. Software-defined Network

Traditional Network



Software-Defined Network



Control and Data Planes

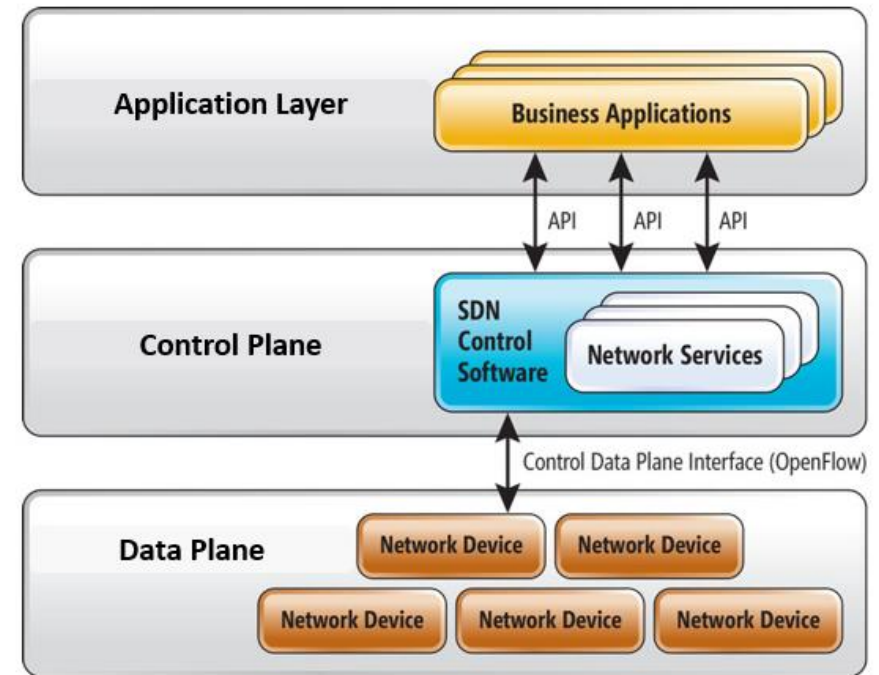
Separate functions that are run on different devices

Control Plane:

- Routing and switching decisions
- Policy enforcement
- Implemented as software in a virtual machine, or as an appliance

Data Plane:

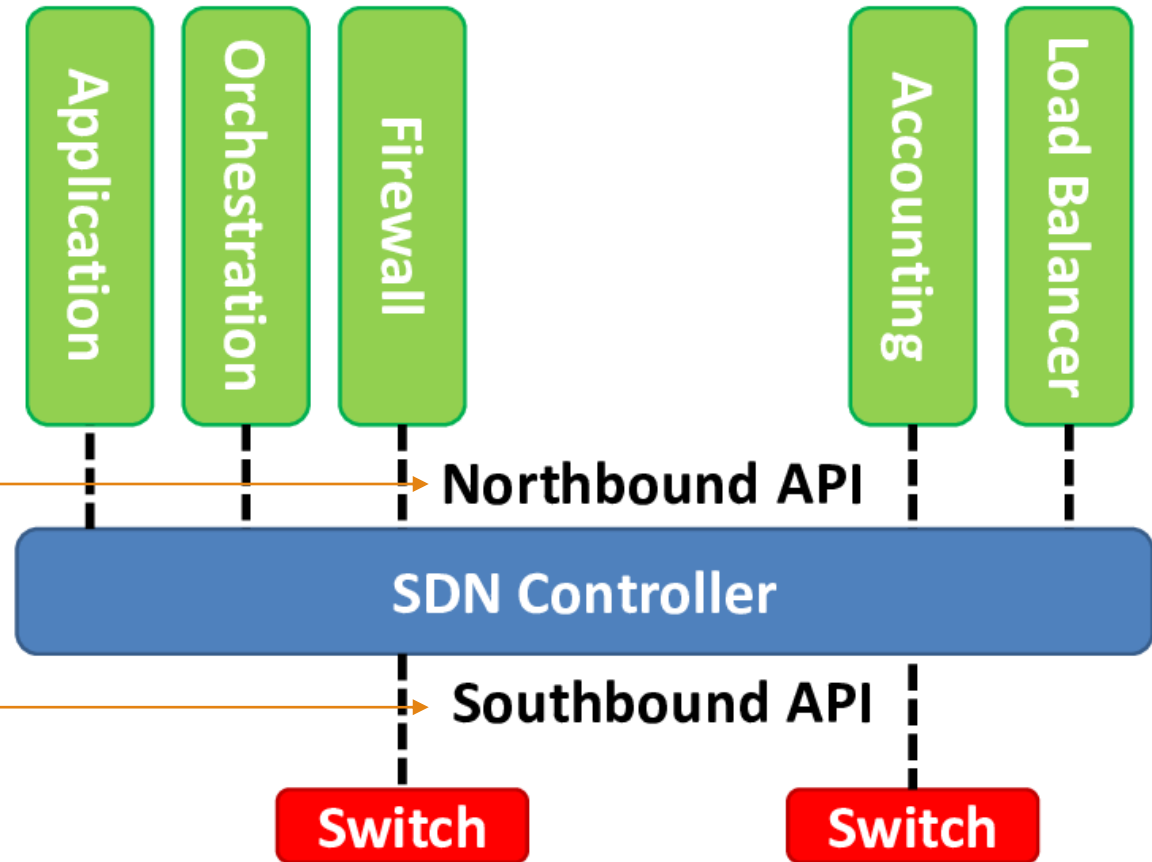
- Actual forwarding of packets
- Executes the decisions made by the control plane
- Implemented as a specialized switch or router



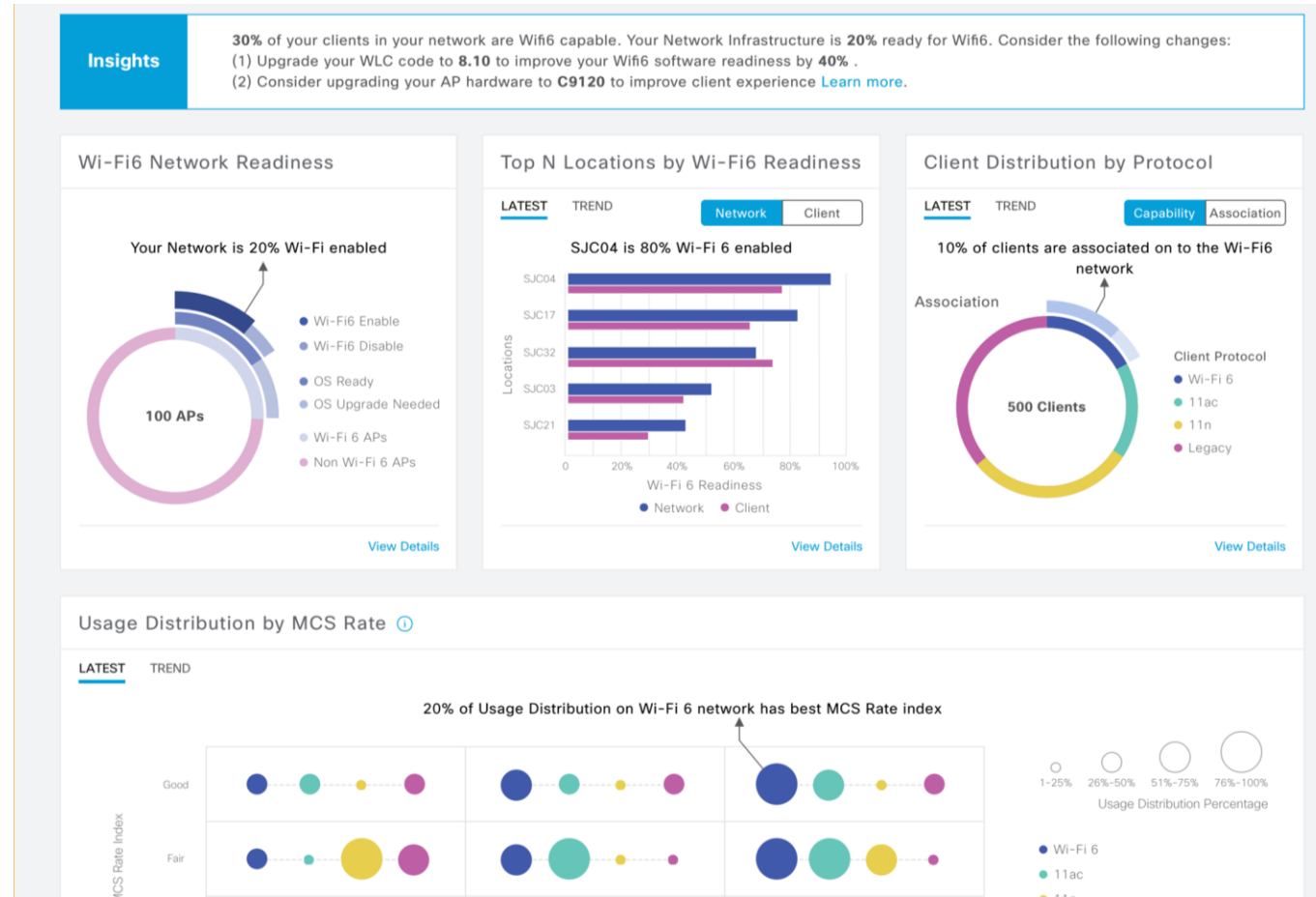
Northbound / Southbound APIs

- Allow SDN Controller to coordinate with automation tools
 - Ansible, Terraform, DNAC, Splunk, VMware, ServiceNow, SolarWinds, etc.
- Typically RESTful APIs

Allow the SDN controller to communicate with and control the hardware devices



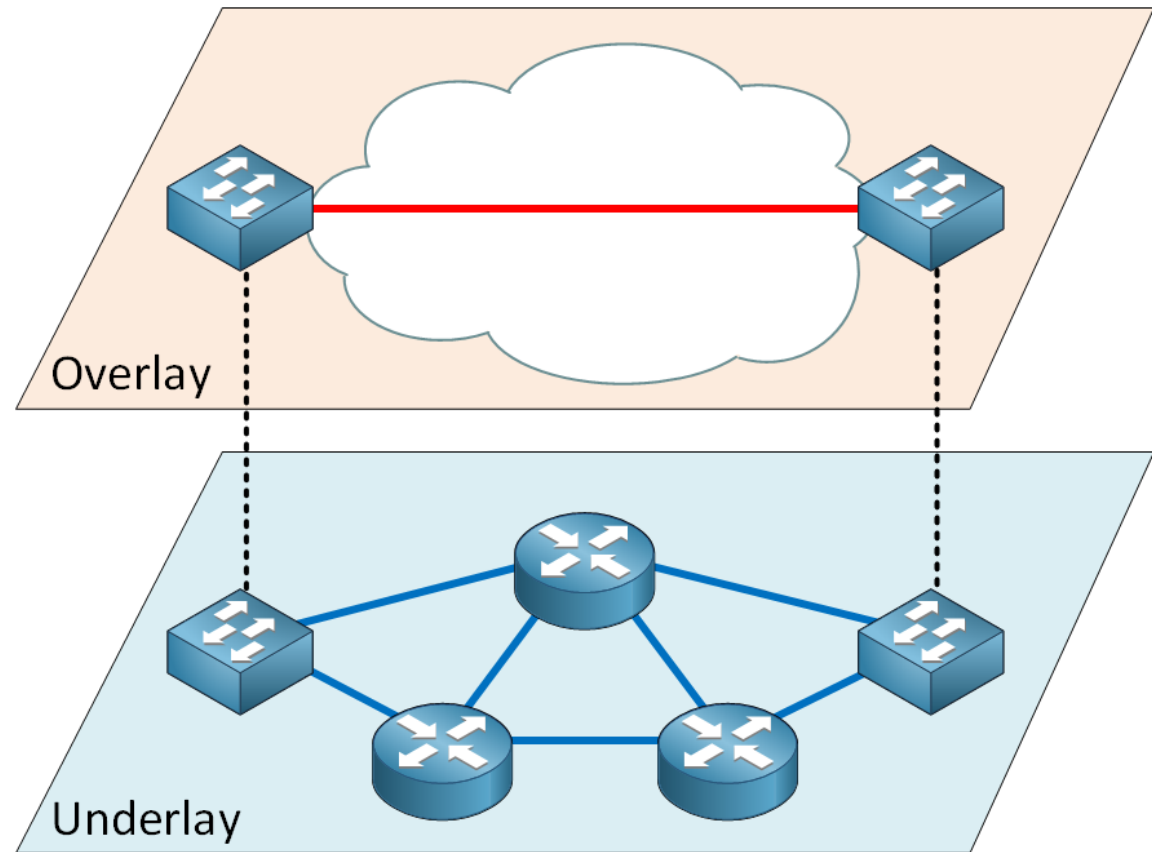
Cisco DNA Center SDN Controller Example



SDN Overlay – Underlay

- A virtual (logical) network built on top of an existing physical network
- Often uses tunneling/VPNs to create the overlay

- The physical infrastructure of specialized routers, switches, and other hardware
- Actually transports the data

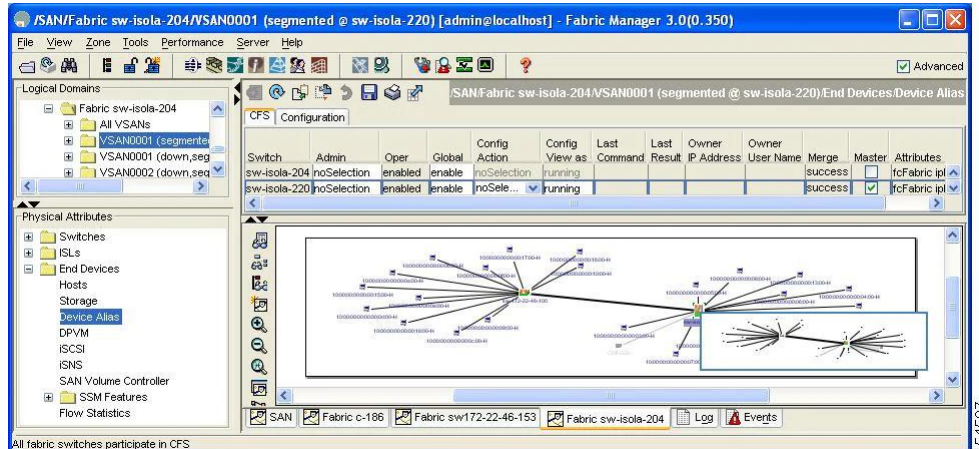


Fabric

A mesh of connections between network devices (access points, switches, routers, etc.)

Acting as one unit, the mesh transports data to its destination

A controller manages both the overlay and the underlay of the fabric



Cisco MDS 9000 Family Fabric Manager



Cisco UCS 6300 Series Fabric Interconnects



Fabric vs. Traditional Architecture

	Fabric	Traditional
Topology	• Spine-leaf • Flatter topology provides consistent latency and bandwidth • Partial mesh • Offers multiple paths	• Three-tier • Single-path nature • Less redundancy
Traffic Pattern	• East-West traffic optimization • Server-to-server • Server-to-storage	• North-South traffic optimization • Traffic predominantly flows between internal and external network
Scalability / Flexibility	• Scale horizontally • Add more leaf & spine switches	• Scaling is more difficult due to hierarchical nature
Management / Automation	• SDN central control	• Manual configuration and management

9.3 JSON

CCNA 200-301 v1.1

Module 9

JSON Overview

JSON Data

JSON Objects

JSON Arrays

JSON Examples

What is JSON?

JavaScript Object Notation

A text-based, lightweight scripting notation for exchanging data

- Easy for humans to read and write
- Easy for machines to parse and generate
- Uses familiar programming conventions, but is independent of any particular programming language

Has many use cases including:

- Web development, mobile apps, software development, network management and automation, database data serialization, IoT, cloud computing, logging and analytics, machine learning and AI



JSON Data

In JSON, the data can be organized in a variety of ways

```
"string": "Hello, World!",  
"number": 42,  
"object": {"key": "value"},  
"array": [1, 2, 3],  
"boolean": true,  
"null": null
```



JSON Objects

JSON Objects are collections of key : value pairs enclosed in curly braces { }

Each key is a string

Each value can be a string, number, Boolean, array, or another object

```
{  
  "hostname": "Router01",  
  "interface": "GigabitEthernet0/1",  
  "status": "up",  
  "description": "Main uplink to ISP"  
}
```

```
{  
  "hostname": "Router1",  
  "enabled": true,  
  "port": 8080  
}
```



JSON Arrays

Arrays are ordered lists of values enclosed in square brackets []

Each value can be a string, number, Boolean, array, or object

```
[  
  "router",  
  "switch",  
  "AP"  
]
```

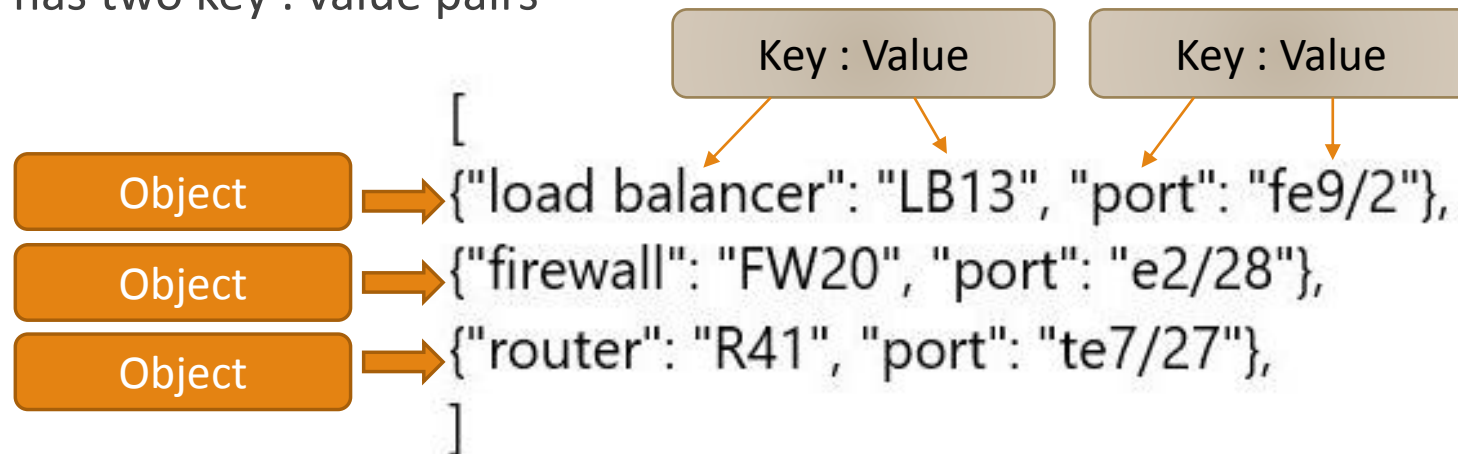


JSON Example #1

This entire data structure is an array

The array is a list of three objects

Each object has two key : value pairs



JSON Example #2

This structure is an object

The object has one key : value pair

The key is “Interfaces”

The value is an array of interface names

```
{  
  "Interfaces":["ethernet0/3", "ethernet0/4", "ethernet0/5"]  
}
```



JSON Example #3

```
{  
  "interfaces": [  
    {  
      "name": "GigabitEthernet0/1",  
      "description": "Uplink"  
    },  
    {  
      "name": "GigabitEthernet0/2",  
      "description": "Downlink"  
    }  
  ]  
}
```

- This object has one key/value pair
- The key is “interfaces”
- The value is an array of two objects
- Each object in the array has two key/value pairs inside:
 - name
 - description

9.4 REST APIs

CCNA 200-301 v1.1

Module 9

REST APIs

REST API Resources

REST API Authentication

REST HTTP Format

CRUD

CRUD Examples

REST API vs CLI

What is an API?

Application Programming Interface

A set of protocols, tools, and definitions that allow software applications to communicate with network devices and services

Usually accessible as a web URL/URI (e.g. <https://api.example.com/users/123>)

Cisco provides a range of APIs

- These enable developers and network administrators to automate tasks, integrate systems, and enhance the functionality of their network infrastructure



What is a REST API?

Representational State Transfer Application Programming Interface

A set of rules and conventions for interacting with network management services over HTTP

Provides a standardized way for applications and tools to communicate with Cisco network devices and services

- Enables programmatic access to device / service functionality

Follows a client-server architecture

Uses HTTP requests to perform CRUD (Create, Read, Update, Delete) operations on resources



REST API Resources

Any network device, service, feature, configuration etc. that can be managed via a REST API call.

Typically represented in JSON format

Can be accessed via standardized URI paths

- Example: `https://<device-ip>/restconf/data/ietf-interfaces:interfaces`



REST API Authentication

Cisco devices can be configured to require authentication to access a resource

You can use an API to authenticate manually or as part of a script

Authentication can be Basic (Base64 encoded password), Token-based, or OAuth-based

Example:

Host: cisco.api.com

GET /api/v1/resource HTTP/1.1

Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=



REST HTTP Format

REST uses HTTP including a header, request body, and response body

Data is typically encoded in JSON format

Headers:

- Content-Type: Specifies the media type of the resource (e.g., application/json).
- Authorization: Contains authentication credentials for the API.

Request Body:

- Used with POST and PUT requests to send data to the server

Response Body:

- Contains the data returned by the server



CRUD

CRUD stands for Create, Read, Update, and Delete

Each is an operation type that can be performed on network resources via a REST API

CRUD operations map to HTTP verbs

The HTTP verbs are used to communicate the actual request to the API

CRUD Operation	HTTP Verb
Create	POST
Read	GET
Update	PUT / PATCH
Delete	DELETE



Create (POST) Example

Add a new resource to the network (network device, VLAN, security policy, etc.)

Example: Host is <https://api.cisco.com>

```
POST /api/v1/devices
```

Client API Call

```
{  
  "hostname": "new-switch",  
  "ip_address": "192.168.1.10",  
  "model": "Cisco Catalyst 9000"  
}
```

Client Request body

```
{  
  "status": "success",  
  "message": "Device added successfully",  
  "device_id": "12345"  
}
```

Server Response



Read (GET) Example

Retrieve information about a resource

- Fetch details / current configuration / statistics about particular device

Example: Host is <https://api.cisco.com>

```
GET /api/v1/devices/12345
```

Client API Call

```
{  
  "device_id": "12345",  
  "hostname": "new-switch",  
  "ip_address": "192.168.1.10",  
  "model": "Cisco Catalyst 9000",  
  "status": "active"  
}
```

Server Response



Update (PUT or PATCH) Example

Modify an existing resource (device configuration, security policy, network settings, etc.)

```
PUT /api/v1/devices/12345
```

Client API Call

```
{  
  "hostname": "updated-switch",  
  "ip_address": "192.168.1.11"  
}
```

Client Request body

```
{  
  "status": "success",  
  "message": "Device updated successfully"  
}
```

Server Response



Delete (DELETE) Example

Remove a resource from the network

- Examples: decommissioned network device, obsolete VLAN, outdated policy

```
DELETE /api/v1/devices/12345
```

Client API Call

```
{  
  "status": "success",  
  "message": "Device deleted successfully"  
}
```

Server Response



REST API vs CLI

The Command Line Interface is effective for manual, one-off configurations and troubleshooting

REST is best for automation and scripting

- More scalable than manually configuring each device via the CLI
- Scripts can be written to configure or monitor multiple switches simultaneously with a single API call
- Ensures consistent configurations across multiple devices, reducing the likelihood of human error
- REST APIs can be integrated with orchestration tools like Ansible, Puppet, or Chef



CLI vs REST Example

The following examples perform the same task on a Cisco Nexus switch:

- Create VLAN 10
- Name VLAN 10 “TestVLAN”
- Set interface Ethernet 1/1 to be an access port
- Assign the interface to VLAN 10



CLI Example

```
ssh admin@switch
SW1# configure terminal
SW1(config)# vlan 10
SW1(config-vlan)# name TestVLAN
SW1(config-vlan)# interface ethernet 1/1
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 10
SW1(config-if)# end
SW1# copy running-config startup-config
```



REST with JSON Example

Create VLAN 10 and name it "TestVLAN"

HTTP Request

POST https://<nexus-ip-address>/api/mo/sys.json

Header

Content-Type: application/json

JSON Payload

```
{  
  "vlanInst": {  
    "attributes": {  
      "id": "10",  
      "name": "TestVLAN"  
    }  
  }  
}
```



REST with JSON Example (cont'd)

Set Interface Ethernet 1/1 as an Access Port and Assign it to VLAN 10

HTTP Request

POST https://<nexus-ip-address>/api/mo/sys/intf.json

Header

Content-Type: application/json

JSON Payload

```
{
  "l1PhysIf": {
    "attributes": {
      "id": "eth1/1",
      "descr": "Set as access port",
      "mode": "access",
      "accessVlan": "10"
    }
  }
}
```



9.5 Management Mechanisms

CCNA 200-301 v1.1

Module 9

Ansible

Terraform

Ansible

Ansible is an open-source automation tool for managing infrastructure as code

Uses playbooks for repeatable configuration tasks

Used for configuration management, application deployment, task automation, and IT orchestration.

- Aims to simplify complex IT processes

Allows users to define infrastructure as code, ensuring consistent and repeatable deployment and management of systems.



Ansible Components

Ansible involves a control node and multiple managed nodes

Control Node:

Ansible is installed on a control node

- Typically a server or a VM running a Linux-based operating system.
- The control node is where Ansible playbooks are executed
- It communicates with network devices over SSH or APIs.

Managed Nodes:

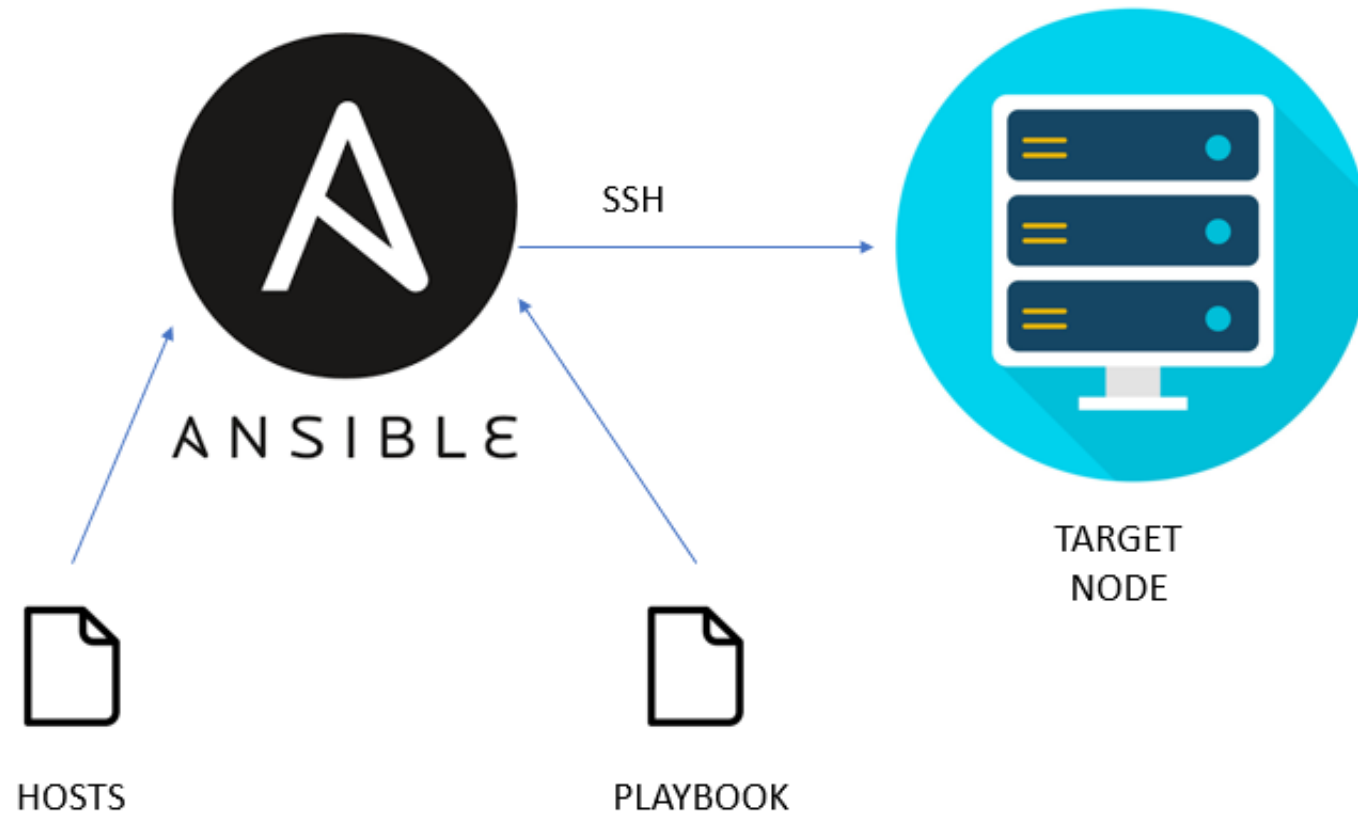
- Ansible operates in an agentless manner
- Does not require any software to be installed on the managed nodes (i.e., the network devices).

Ansible supports a wide range of Cisco network devices:

- IOS, IOS-XE, IOS-XR, NX-OS, and ASA devices



Ansible Example



Terraform

An open-source infrastructure as code (IaC) tool

Scripts use a declarative configuration language to define and provision the infrastructure

- Ensures consistent and repeatable deployments

Uses a “provider” to talk to the target platform or service

- Automate the provisioning of Cisco network devices, such as switches, routers, and access points.

Cisco products that can act as a Terraform provider:

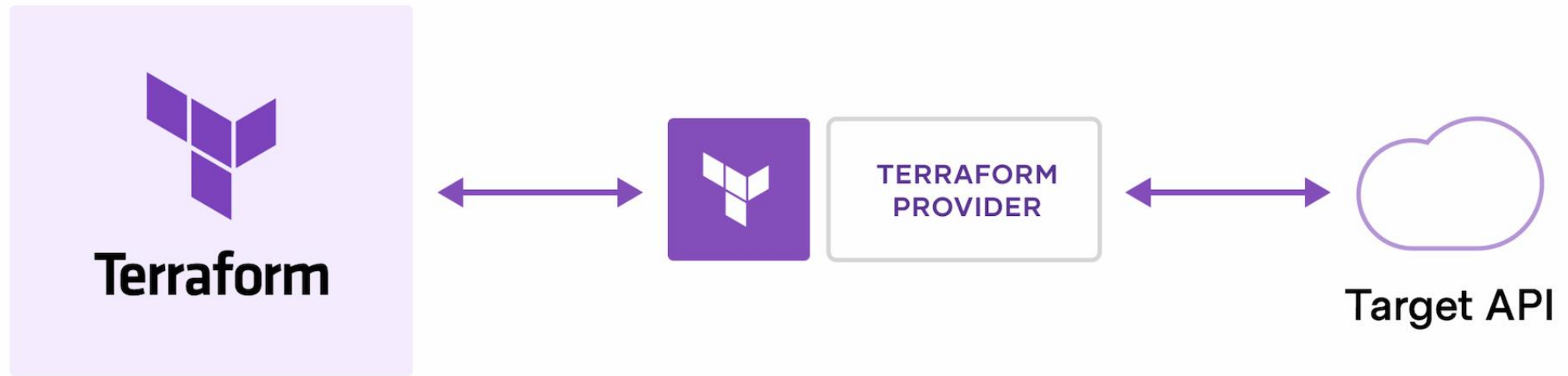
- Cisco ACI, Cisco Meraki, Cisco NSO, Cisco ASA, Cisco SD-WAN

Cisco devices and OSeS that provide APIs to allow communications with a Terraform provider:

- Nexus 9000 series switches, ASA 5500-X series firewalls, IOS XE, IOS XR, NX-OS



Terraform Example



9.6 AI in Network Operations

CCNA 200-301 v1.1

Module 9

Predictive AI

Generative AI

Machine Learning

Predictive AI

Designed to forecast future events, behaviors, and conditions based on historical and real-time data analysis

Used for forecasting, preventive maintenance, and anticipating future demands and threats

Provides foresight into potential future issues and conditions, allowing for proactive measures to be taken



Generative AI

An advanced technological approach that leverages machine learning and artificial intelligence to enhance network management, automation, and optimization

Focuses on creating new data, configurations, and solutions based on learned patterns from existing data

Provides immediate solutions and optimizations, automating tasks and enhancing network operations dynamically



Machine Learning

The foundational technology that powers both generative AI and predictive AI in Cisco network operations

ML algorithms ingest vast amounts of historical network data, including configurations, traffic logs, performance metrics, and security events

These algorithms learn patterns and relationships within the data, identifying optimal configurations, common traffic routes, and typical responses to network issues



Examples of Cisco Products that Use AI

Cisco DNA Center:

- A comprehensive network management and command center for wired and wireless networks

Cisco AI Network Analytics:

- A component of the Cisco DNA Center that leverages artificial intelligence and machine learning

SecureX

- Security visibility, threat response, incident management

Cisco Webex

- Video conferencing, messaging and calling

Cisco ThousandEyes

- Visibility into network performance across the Internet, cloud and enterprise networks

Cisco Meraki

- Cloud-management of wireless, switching, security, SD-WAN and smart cameras

Cisco Stealthwatch

- Network visibility and security analytics for detecting threats and managing incidents

Cisco Tetration

- Data center security and analytics platform

Cisco AppDynamics

- Real-time monitoring and analytics for applications

Cisco Intersight

- Cloud-based management platform for Cisco and third-party infrastructure

Cisco Umbrella

- Cloud-delivered security for DNS, web gateway firewalls, Cloud Access Security Broker (CASB) functionality



Review

9.7 Review

CCNA 200-301 v1.1

Module 9

Review

Network Management Automation is the use of software and tools to automate the configuration, monitoring, management and maintenance of network devices and infrastructure

Infrastructure as code is the managing and provisioning computer data centers through the use of machine readable definition files

In a traditional network the control and data planes coexist together on each Network device

Each network device is configured and managed separately

In a Software Defined Network (SDN) the control plane is separated from the data plane

- A central software manager makes forwarding decisions, telling the underlying devices what to do



Review (cont'd)

A Northbound API allows an SDN controller to coordinate with automation tools such as Ansible, Terraform and Cisco DNS

A Southbound API allows the SDN controller to communicate with and control the hardware devices

In an SDN, you will have a virtual or logical network overlay on top of the physical infrastructure underlay

The overlay often uses tunneling or VPNs to create the logical topology

A fabric is a mesh of connections between network devices that all act as a single unit

A controller manages both the overlay and the underlay of the fabric



Review (cont'd)

JSON is a text-based lightweight scripting notation for exchanging data

JSON has various ways of organizing data including strings, numbers, objects, arrays, Booleans, and nulls

A JSON string is text surrounded by quotes **"some data"**

A JSON number is just a number **42**

A JSON object is a key : value pair enclosed in curly braces **{name : SW1}**

A JSON array is a list of items enclosed in square brackets **["R1", "R2", "R3"]**

A JSON Boolean is either true or false **true**

A JSON null is a null value **null**

In a JSON object, the key is always a string, but the value can be any data type including another object



Review (cont'd)

An application programming interface (API) is a set of protocols tools and definitions that allow software applications to communicate with network devices and services

A REST API Is a set of rules and conventions for interacting with network Management Services over http

REST Uses HTTP requests to perform CRUD (create, read, update, delete) on resources

The data in a REST request or response is usually encoded in JSON format

The REST CREATE operation uses the HTTP POST verb

The REST READ operation uses the HTTP GET verb

The REST UPDATE operation uses either the HTTP PUT or PATCH verb

The REST DELETE operation uses the HTTP DELETE verb



Review (cont'd)

On a Cisco device, using the CLI is effective for manual, one-off configurations and troubleshooting

Many Cisco devices support REST Communications

- REST is better for Automation and scripting

Ansible is an open source automation tool for managing infrastructure as code

- It uses playbooks for repeatable configuration tasks

Terraform is also an open source infrastructure as code tool

- It uses scripts with declarative configuration language to define and provision the infrastructure



Review (cont'd)

Predictive AI is designed to forecast future events behaviors and conditions based on historical and real-time data analysis

- It provides foresight into potential future issues and conditions, allowing for proactive measures to be taken

Generative AI leverages machine learning and artificial intelligence to enhance network management automation and optimization

- It focuses on creating new data, configurations, and solutions based on learned patterns from existing data

Generative AI provides immediate solutions and optimizations, automating tasks and enhancing network operations dynamically

Machine learning is the foundational technology that powers both generative AI and predictive AI in Cisco network operations

